DSA UNIT IV – TREE

TOPIC: Modified AVL Imbalance Detection Using C++

1. PROBLEM STATEMENT

Write a C++ program to detect LL, RR, LR, and RL imbalances during AVL tree insertion, identify the rotation required, and display the balanced tree.

2. OBJECTIVES

1. To understand AVL trees and their balancing operations.

2. To detect LL, RR, LR, and RL imbalances during insertion.

3. To display the first unbalanced ancestor and its balance factor.

4. To record rotation types and affected node values.

5. To test all four rotation cases and verify the final AVL tree balance conditions.

3. INTRODUCTION

An AVL tree is a self-balancing Binary Search Tree (BST). It maintains balance after every insertion by performing rotations whenever an imbalance occurs.

The balance factor of a node is calculated as:

Balance Factor = Height of Left Subtree - Height of Right Subtree

For a balanced AVL tree, the balance factor of every node must be -1, 0, or +1.

AVL trees provide efficient searching, insertion, and deletion operations.

4. TYPES OF AVL IMBALANCES

LL Case:

This occurs when a node is inserted into the left subtree of the left child. A right rotation is performed.

RR Case:

This occurs when a node is inserted into the right subtree of the right child. A left rotation is performed.

LR Case:

This occurs when a node is inserted into the right subtree of the left child. A left rotation followed by a right rotation is performed.

RL Case:

This occurs when a node is inserted into the left subtree of the right child. A right rotation followed by a left rotation is performed.

5. MODIFICATIONS IMPLEMENTED

Modification 1: First Unbalanced Ancestor

The program displays the first unbalanced ancestor encountered during insertion and its balance factor.

Modification 2: Rotation Details

The program records the rotation type and the values of the affected nodes.

Modification 3: Testing and Balance Verification

The program tests LL, RR, LR, and RL rotation cases. It verifies that the final AVL tree satisfies the balance-factor condition at every node.

6. ALGORITHM

1. Start the program.

2. Create an empty AVL tree.

3. Enter the number of nodes.

4. Enter the values to be inserted.

5. Insert each value using Binary Search Tree insertion rules.

6. Update the height of each node.

7. Calculate the balance factor of each node.

8. Detect the first unbalanced ancestor.

9. Display the unbalanced node and its balance factor.

10. Identify the imbalance type.

11. Perform the required rotation.

12. Record the rotation type and affected node values.

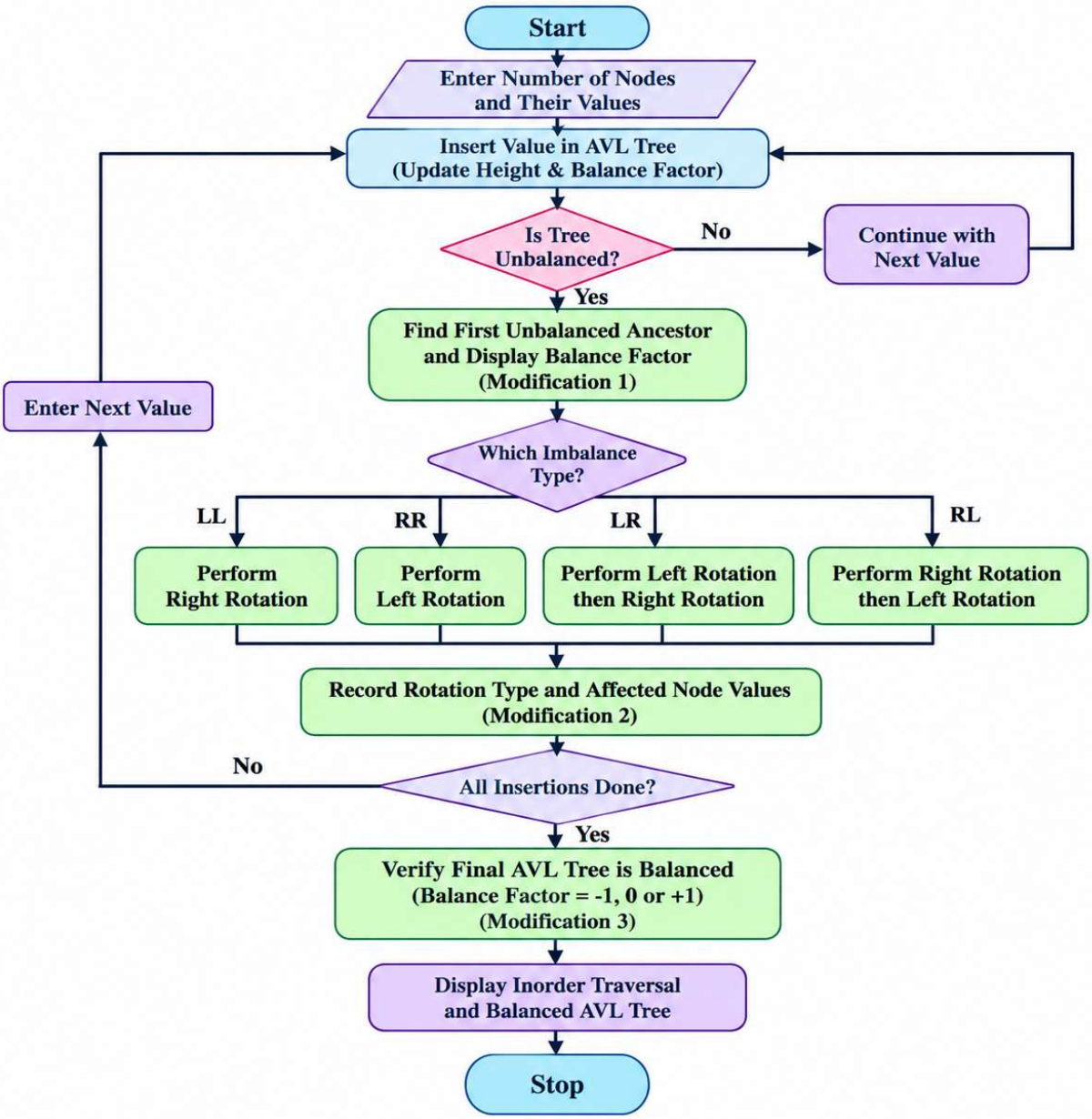
13. Display the inorder traversal and balanced AVL tree.

14. Verify the balance conditions of all nodes.

15. Display the verification result.

16. Stop the program.

7. FLOWCHART



8. SOURCE CODE

```
#include <iostream>

#include <algorithm>

using namespace std;

struct Node

{

    int data;

    int height;

    Node* left;
```

```
    Node* right;
    Node(int value)
    {
        data = value;
        height = 1;
        left = NULL;
        right = NULL;
    }
};
int height(Node* node)
{
    if (node == NULL)
        return 0;
    return node->height;
}
int balanceFactor(Node* node)
{
    if (node == NULL)
        return 0;
    return height(node->left) - height(node->right);
}
Node* rightRotate(Node* y)
{
    Node* x = y->left;
    Node* temp = x->right;
    cout << "Rotation: Right Rotation" << endl;
    cout << "Affected nodes: " << y->data << " and " << x->data << endl;
    x->right = y;
```

```
    y->left = temp;
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;
}
Node* leftRotate(Node* x)
{
    Node* y = x->right;
    Node* temp = y->left;
    cout << "Rotation: Left Rotation" << endl;
    cout << "Affected nodes: " << x->data << " and " << y->data << endl;
    y->left = x;
    x->right = temp;
    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));
    return y;
}
Node* insert(Node* root, int value)
{
    if (root == NULL)
        return new Node(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else
    {
        cout << "Duplicate value ignored: " << value << endl;
```

```
    return root;
}
root->height = 1 + max(height(root->left), height(root->right));
int bf = balanceFactor(root);
// Modification 1: Detect first unbalanced ancestor
if (bf > 1 || bf < -1)
{
    cout << "\nFirst unbalanced ancestor: "
         << root->data << endl;
    cout << "Balance factor: " << bf << endl;
    // LL Case
    if (bf > 1 && value < root->left->data)
    {
        cout << "Imbalance type: LL" << endl;
        return rightRotate(root);
    }
    // RR Case
    if (bf < -1 && value > root->right->data)
    {
        cout << "Imbalance type: RR" << endl;
        return leftRotate(root);
    }
    // LR Case
    if (bf > 1 && value > root->left->data)
    {
        cout << "Imbalance type: LR" << endl;
        root->left = leftRotate(root->left);
        return rightRotate(root);
```

```
        }
        // RL Case
        if (bf < -1 && value < root->right->data)
        {
            cout << "Imbalance type: RL" << endl;
            root->right = rightRotate(root->right);
            return leftRotate(root);
        }
    }
    return root;
}
// Modification 3: Verify AVL balance conditions
bool verifyAVL(Node* root)
{
    if (root == NULL)
        return true;
    int bf = balanceFactor(root);
    if (bf < -1 || bf > 1)
        return false;
    return verifyAVL(root->left) && verifyAVL(root->right);
}
void inorder(Node* root)
{
    if (root == NULL)
        return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
```

```
}
void displayTree(Node* root, int space = 0)
{
    if (root == NULL)
        return;
    space += 5;
    displayTree(root->right, space);
    cout << endl;
    for (int i = 5; i < space; i++)
        cout << " ";
    cout << root->data << endl;
    displayTree(root->left, space);
}
void deleteTree(Node* root)
{
    if (root == NULL)
        return;
    deleteTree(root->left);
    deleteTree(root->right);
    delete root;
}
int main()
{
    Node* root = NULL;
    int n, value;
    cout << "MODIFIED AVL IMBALANCE DETECTION" << endl;
    cout << "Enter number of nodes: ";
    cin >> n;
```

```
    if (n <= 0)
    {
        cout << "Please enter a positive number of nodes." << endl;
        return 0;
    }
    cout << "Enter " << n << " values:" << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> value;
        root = insert(root, value);
    }
    cout << "\nInorder Traversal: ";
    inorder(root);
    cout << "\n\nBalanced AVL Tree:";
    displayTree(root);
    cout << "\nAVL Balance Verification: ";
    if (verifyAVL(root))
        cout << "PASS - Tree is balanced." << endl;
    else
        cout << "FAIL - Tree is not balanced." << endl;
    deleteTree(root);
    return 0;
}
```

9. TEST CASES

Test Case 1: LL Rotation

Input:

3

30 20 10

Expected Result:

Imbalance Type: LL

Rotation: Right Rotation

Inorder Traversal: 10 20 30

AVL Verification: PASS

Test Case 2: RR Rotation

Input:

3

10 20 30

Expected Result:

Imbalance Type: RR

Rotation: Left Rotation

Inorder Traversal: 10 20 30

AVL Verification: PASS

Test Case 3: LR Rotation

Input:

3

30 10 20

Expected Result:

Imbalance Type: LR

Rotations: Left Rotation followed by Right Rotation

Inorder Traversal: 10 20 30

AVL Verification: PASS

Test Case 4: RL Rotation

Input:

3

10 30 20

Expected Result:

Imbalance Type: RL

Rotations: Right Rotation followed by Left Rotation

Inorder Traversal: 10 20 30

AVL Verification: PASS

Test Case 5: Single Node

Input:

1

50

Expected Result:

No imbalance detected.

Inorder Traversal: 50

AVL Verification: PASS

10. SAMPLE OUTPUT

```
MODIFIED AVL IMBALANCE DETECTION
Enter number of nodes: 3
Enter 3 values:
30 20 10

First unbalanced ancestor: 30
Balance factor: 2
Imbalance type: LL
Rotation: Right Rotation
Affected nodes: 30 and 20

Inorder Traversal: 10 20 30

Balanced AVL Tree:
     30

20

     10

AVL Balance Verification: PASS - Tree is balanced.

--------------------------------
Process exited after 19.37 seconds with return value 0
Press any key to continue . . .
```

11. COMPLEXITY ANALYSIS

Time Complexity:

Insertion: O(log n) for an AVL tree because the tree remains balanced.

Searching: O(log n) because the height of the AVL tree is logarithmic.

Balance Verification: O(n) because every node may need to be checked.

Space Complexity:

O(n) for storing n nodes in the AVL tree.

12. APPLICATIONS

1. Database indexing: AVL trees can help maintain sorted data for efficient searching.

2. Searching applications: They provide efficient search operations.

3. Dictionary implementations: They can store keys in sorted order.

4. Data management: They maintain balance when values are inserted dynamically.

13. CONCLUSION

The modified AVL Imbalance Detection program identifies LL, RR, LR, and RL imbalances during insertion and performs the required rotations.

It also displays the first unbalanced ancestor, records rotation details, and verifies the balance conditions of the final tree.

Testing all four rotation cases demonstrates how AVL trees maintain balance and support efficient operations.